

2.0.1 Kontrollstrukturen / Algorithmische Bausteine

Die Klasse `Turtle` stellt ein bewegbares Dreieck dar, welches seinen abgelaufenen Weg wie mit einem Stift nachzeichnet.

Aufgabe 0) Predict

Zeichne das Ergebnis des Java-Codes auf ein Blatt auf.

HAUPTPROGRAMM:

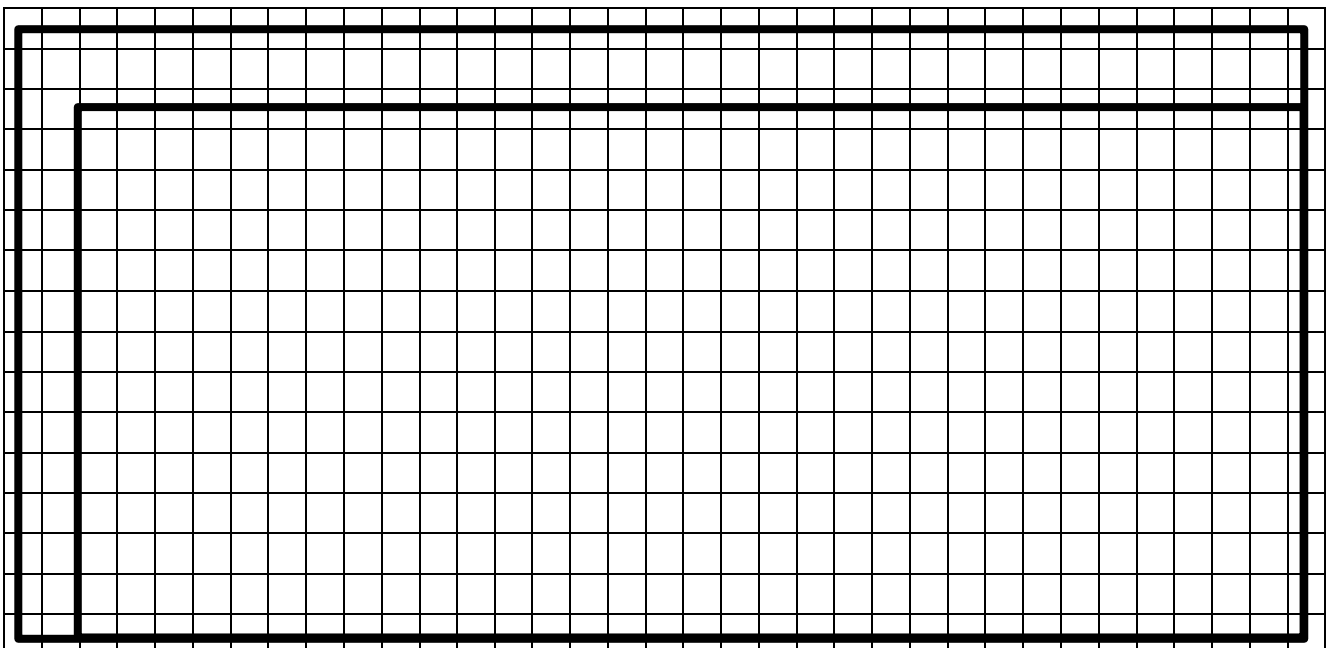
```
new World(800, 600);  
Zeichner z = new Zeichner(400, 300);  
  
z.quadrat(200);  
z.turn(-45);  
z.quadrat(200);
```

```
class Zeichner extends Turtle {  
  
    Zeichner(int x, int y) {  
        super(x, y, true);  
        setBorderColor(Color.aqua);  
        setBorderWidth(5);  
        penDown();  
    }  
  
    void quadrat(double seitenlänge) {  
        for (int i = 0; i < 4; i++) {  
            forward(seitenlänge);  
            turn(90); //dreht gegen Uhrzeigersinn  
        }  
    }  
}
```

Aufgabe 1) OnlineIDE Projekt QUADRATROSEN

- Führe das Programm aus und überprüfe deine Vermutung.
- Ändere die Methode so ab, dass verschiedene Vielecke entstehen.
- Ergänze die Methode `quadratRose(anzahl)`.
- Zeichne das Struktogramm zu deiner Methode OHNE die Methode `quadrat(...)` -> Struktogramm im Struktogramm.

Name der Kontrollstruktur: _____



Wiederholung mit fester Anzahl in bedingte Wiederholung umwandeln

Wiederholung mit fester Anzahl	Wiederholung mit Bedingung
<pre>for (int i = 0; i < 3; i = i + 1) { erstelleQuadratAn(i, 1); }</pre>	<pre>int i = 0; while (i < 3) { erstelleQuadratAn(i, 1); i = i + 1 }</pre>

Wandle folgende Wiederholungen in die jeweils andere (in Java) um.

```
int i = 15;
while (i > 0){
    println(i);
    i = i-2;
}
```

```
for(int a = 5; a < 15; a++) {
    dingeTun();
}
```

Aufgabe 2) OnlineIDE Projekt CURVINGFEVERS

- Führe das Programm aus.
- Untersuche die Klassen `Curver1` und `Curver2`. Was fällt dir auf?
- Ergänze die Methoden passend.
- Übertrage deinen Code auf das Arbeitsblatt und markiere die Bestandteile.
- Ergänze die Methode `crashPrüfen()` in `Curver1` und `Curver2`.

Name der Kontrollstruktur: _____

Bedingung	
wahr	falsch
Sequenz1	Sequenz2

Bedingungen in Java

				istWand()
größer	kleiner oder gleich	gleich	ungleich	

Bedingungen können auch kombiniert werden: _____ -> **|** | ; _____ -> **&&**